

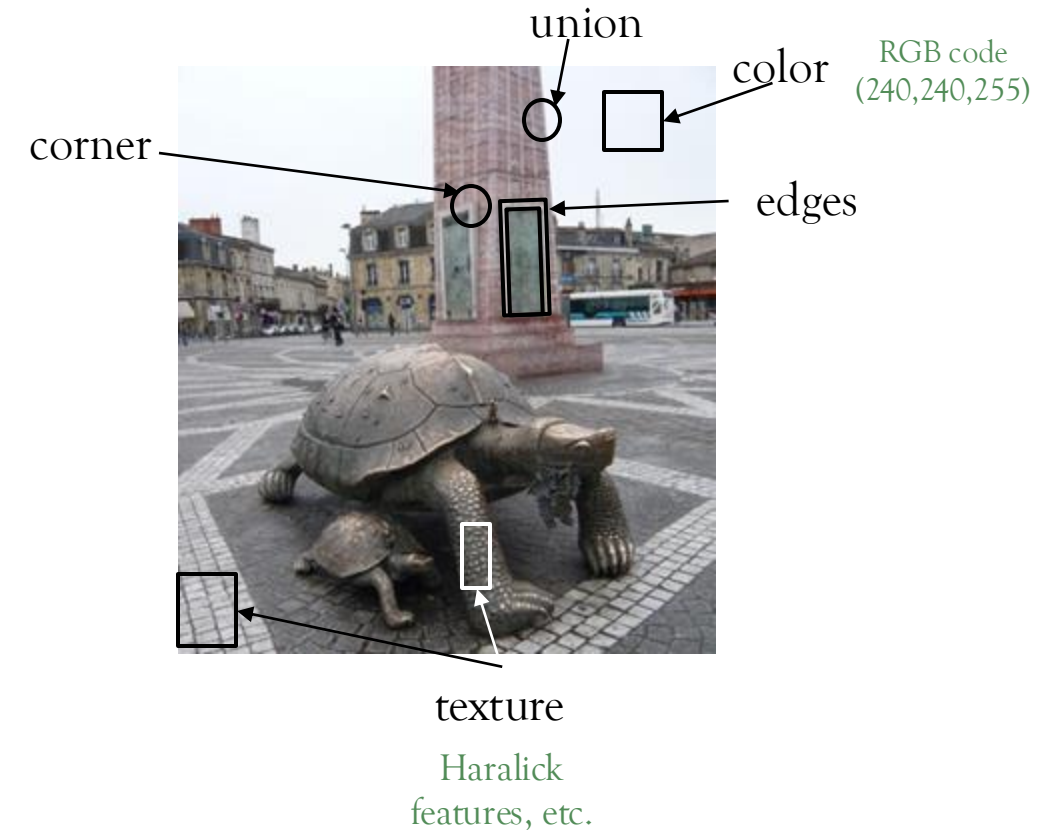


5. FEATURE DESCRIPTORS

1. *Statistical region descriptors*
2. *Invariant measures*

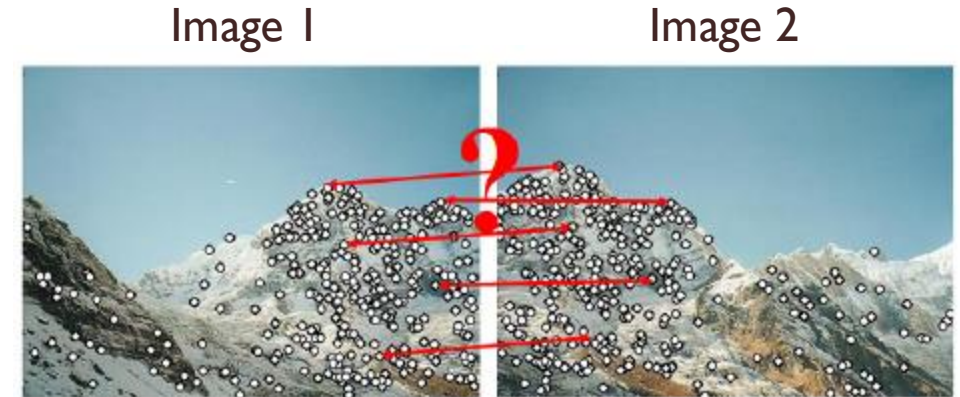
FEATURE DESCRIPTORS

Features	Feature descriptors (examples)
Brightness	average + standard deviation; maximum; minimum
Color	color image components
Geometric / shape	Size, eccentricity, compactness
Texture	Haralick features
...	...



INVARIANT MEASURES

- **Local appearance** of features will change in orientation & scale, even undergo affine (translation, rotation...) deformations
- Compensate for these changes → extracting local scale, orientation, or affine frame estimate & resample
- Local appearance of image patches will usually **still vary** from image to image



- Our aim is to use **descriptors** that are **invariant** to such changes while **preserving the ability to discriminate** between different (non-corresponding) patches
- View-invariant local image descriptors:
 - **Scale invariant feature transform (SIFT)**

SIFT: SCALE INVARIANT FEATURE TRANSFORM

- Algorithm developed by Lowe [2004] for extracting **invariant features** from an image
- It **transforms** image data into scale-invariant coordinates relative to local image features
- Strongly **heuristic**, it uses a high number of experimentally determined parameters
- **SIFT features** (called **keypoints**) are **invariant to scale and rotation**, and are robust across a range of affine distortions, changes in 3D viewpoint, noise, and changes of illumination
- **Input:** an image
- **Output:** n-dimensional feature vector whose elements are the invariant feature descriptors



SIFT: SCALE INVARIANT FEATURE TRANSFORM

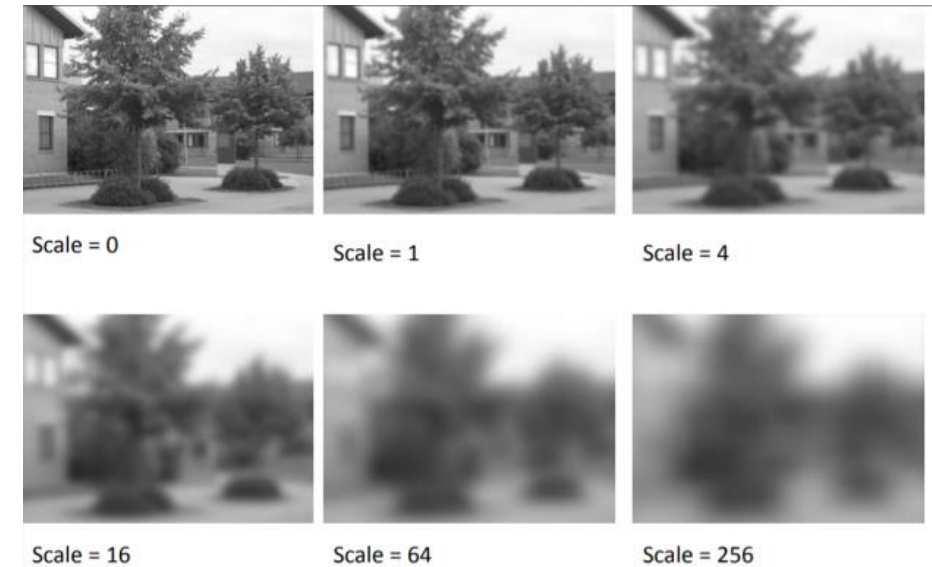
Main 4 steps:

1. Construct the **scale space**
 2. **Keypoint** localization (+ improve the accuracy of keypoints locations, + delete unsuitable keypoints)
 3. Compute keypoint **orientations**
 4. Compute keypoint **descriptors**
-
5. **Keypoint Matching** (if matching is required)

SIFT: SCALE INVARIANT FEATURE TRANSFORM

Scale Space: how scale invariance is achieved by SIFT

- First stage: find **image locations** that are **invariant** to scale change
 - This is achieved by searching for stable features across all possible scales, using a function known as scale space
 - The idea is to have a formalism to work with all relevant scales simultaneously.
- Scale space function represents an image as a **one-parameter family of smoothed image**. The parameter controlling the smoothing is the scale parameter
- In SIFT, Gaussian kernels are used to implement smoothing → the **scale parameter** is the **standard deviation**



<https://theailearner.com/2021/10/12/sift-scale-space-extrema-detection/>

As we **increase** the scale, the fine details will be lost, and only coarser information prevails

SIFT: SCALE INVARIANT FEATURE TRANSFORM

Scale Space: how scale invariance is achieved by SIFT

- The **scale space** of an image is a function $L(x, y, \sigma)$ that is the result of **convolving** the image f with a Gaussian kernel at different scales, $G(x, y, \sigma)$

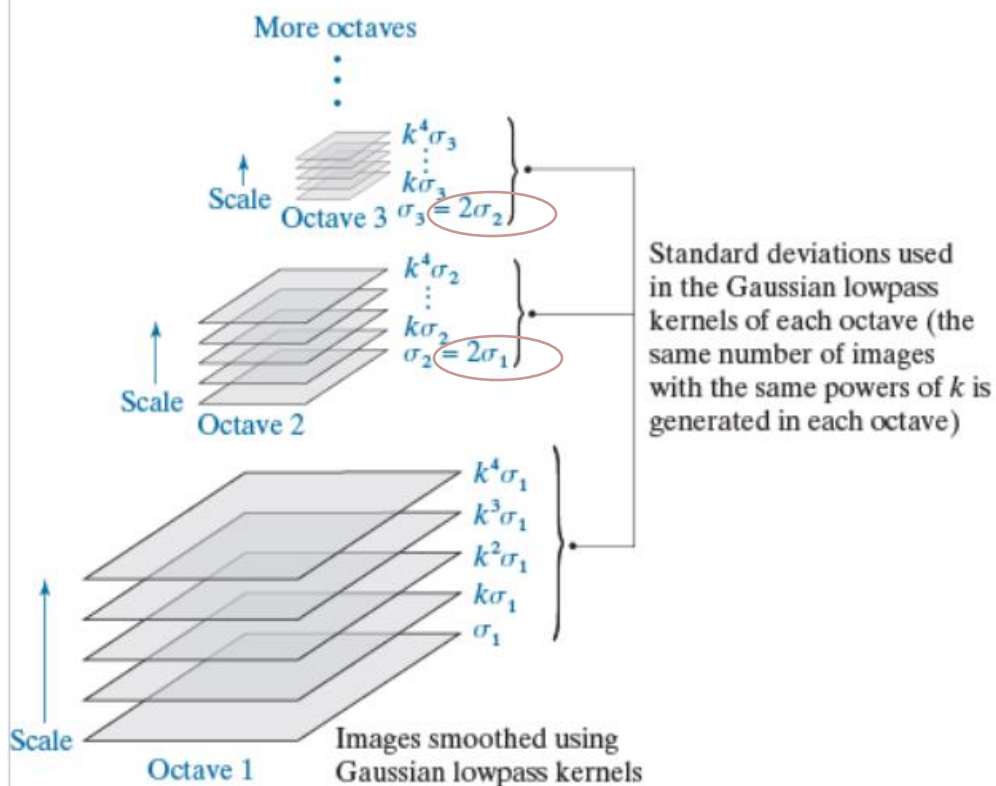
$$L(x, y, \sigma) = G(x, y, \sigma) \star f(x, y)$$

- The scale is controlled by the parameter σ , and G is of the form:

$$G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)/2\sigma^2}$$

- The input image $f(x, y)$ is successively convolved with Gaussian kernels having standard deviations $\sigma, k\sigma, k^2\sigma, k^3\sigma, \dots$ to generate a “**stack**” of Gaussian-filtered (**smoothed/blurred**) **images** that are separated by a constant factor, k

SIFT: SCALE INVARIANT FEATURE TRANSFORM

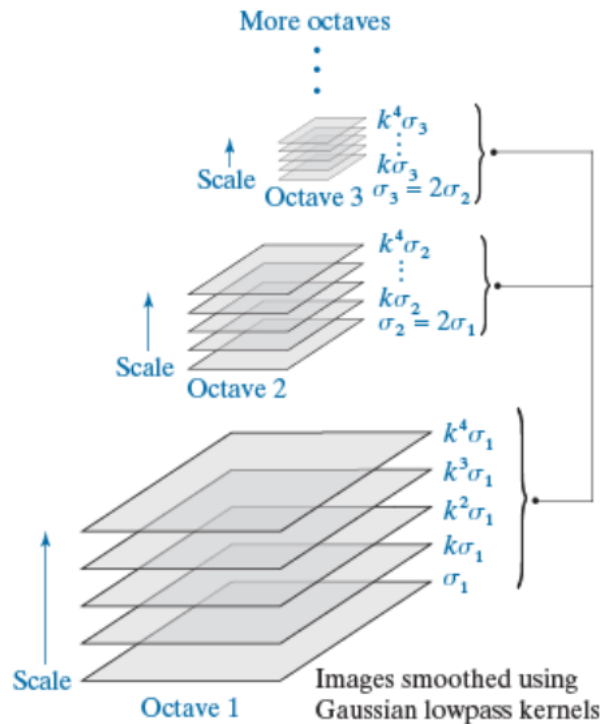


Space parameter σ

Interval $s=2 \rightarrow \#$ of images is $s+1+2=5$

- SIFT subdivides the scale space into **octaves**, with each octave corresponding to a **doubling of σ**
- It further subdivides each octave into s (integer number) intervals
- The value used in the Gaussian kernel that generates the image corresponding to an octave is $k^s \sigma = 2\sigma$ which means that $k = 2^{1/s}$
- Example:** for $s=2 \rightarrow k = \sqrt{2}$. The input image is successively smoothed using standard deviations of:
 - $k^0 \sigma = \sigma$
 - $k^1 \sigma = \sqrt{2} \sigma,$
 - $k^2 \sigma = 2\sigma \rightarrow$ next octave
- This indicates that the number of smoothed images generated in an octave is $s+1$.
- But, in order to compute differences of Gaussians (next step) covering a full octave, two additional images past the octave image are required, giving a total of $s+3$ images

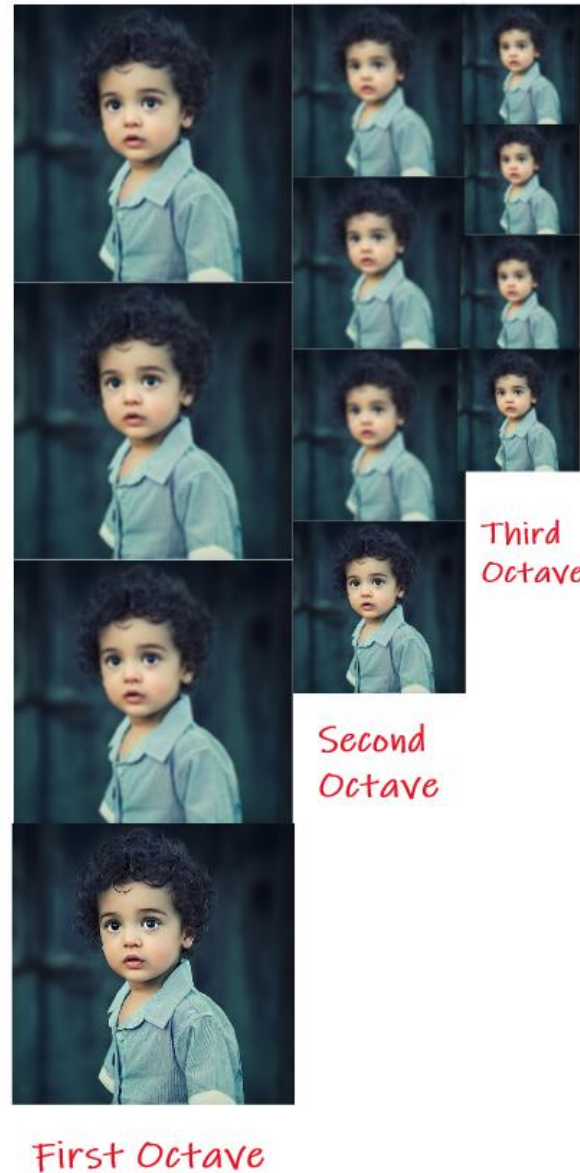
SIFT: SCALE INVARIANT FEATURE TRANSFORM



1st image: $k^0 \sigma = \sigma$
2nd image: $k^1 \sigma = (\sqrt{2}) \sigma$,
3rd image: $k^2 \sigma = (\sqrt{2})^2 \sigma = 2\sigma \rightarrow$
 $\rightarrow$ **OCTAVE IMAGE** (1st image of next octave)

- **Second octave:**
 - The first image is computed by:
 - **downsampling** the original image to **half size**
 - smoothing it using a kernel **$\sigma_2 = 2\sigma_1$**
 - Subsequent images are smoothed using σ_2 , with the same sequence of values of k as in the first octave
- Same procedure is repeated for subsequent octaves
- Note that, when $k = \sqrt{2}$ (i.e. 5 images per octave):
 - 1st image of octave 2 == 3rd image from the top of octave 1 but downsampled
 - This is true for any octave
 - This image is called the **octave image** because the sigma used to smooth it, is twice the value of the sigma used to smooth the first image in the octave

SIFT: EXAMPLE

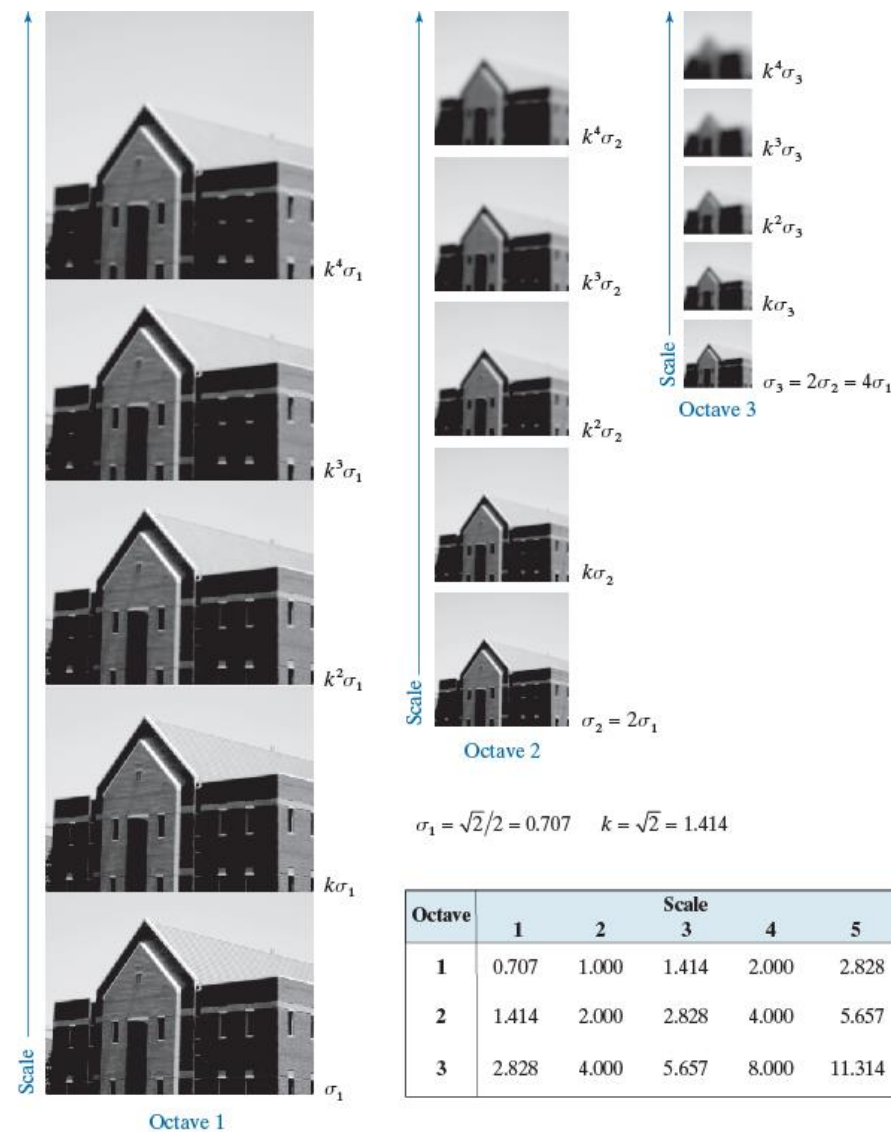


<https://theailearner.com/2021/10/12/sift-scale-space-extrema-detection/>

SIFT: SCALE INVARIANT FEATURE TRANSFORM

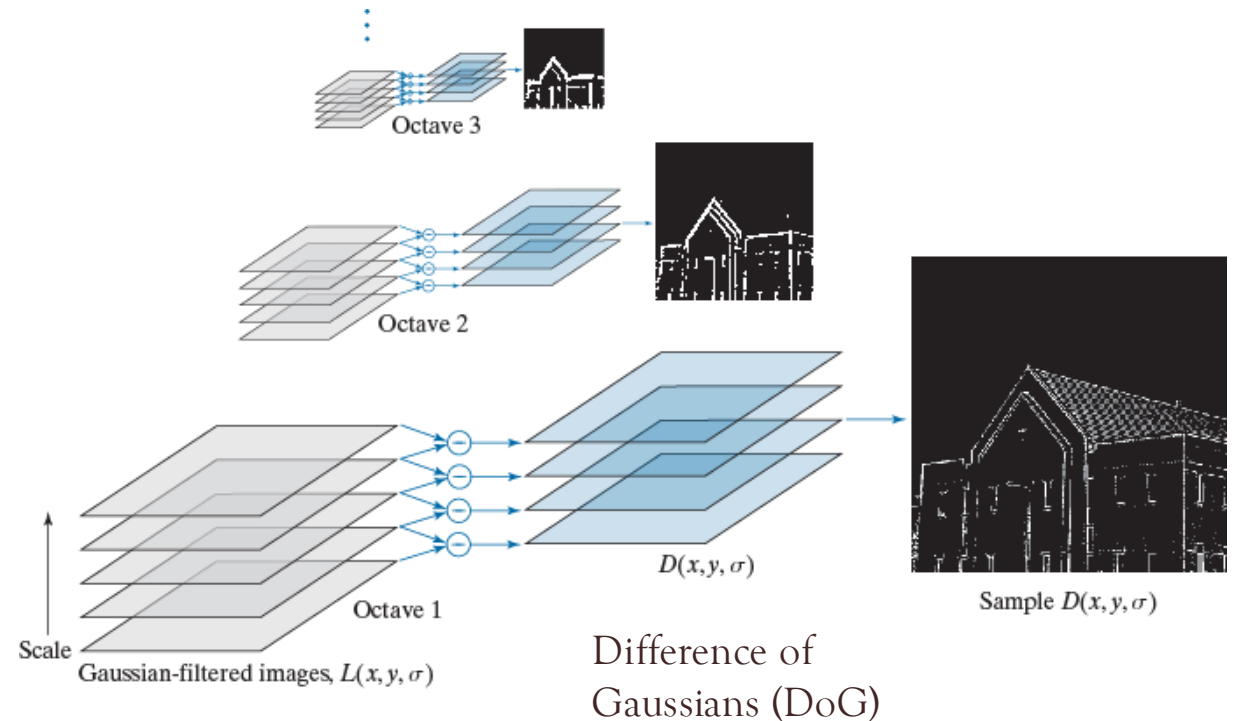
Scale space: example with numbers

- As each octave is composed of 5 images, we are using $s=2$.
- We chose $\sigma_1 = \sqrt{2}/2 = 0.707$
- Then (remember $k = 2^{1/s} \rightarrow k = \sqrt{2} = 1.414$)
- Octave 1:
 - $\sigma_1 = \sqrt{2}/2 = 0.707$
 - $k \sigma_1 = \sqrt{2} \sqrt{2}/2 = 1.000$
 - OCTAVE IMAGE: $k^2 \sigma_1 = 2 \sqrt{2}/2 = 1.414$
 - $k^3 \sigma_1 = 2 \sqrt{2} \sqrt{2}/2 = 2.000$
 - $k^4 \sigma_1 = 4 \sqrt{2}/2 = 2.828$
- Octave 2:
 - $\sigma_2 = 2 \sigma_1 = 2 \sqrt{2}/2 = 1.414$
 - $k \sigma_2 = \sqrt{2} 2 \sqrt{2}/2 = 2.000$
 - OCTAVE IMAGE: $k^2 \sigma_2 = 2.828$
 - $k^3 \sigma_2 = 4.000$
 - $k^4 \sigma_2 = 5.657$



SIFT: SCALE INVARIANT FEATURE TRANSFORM

- Use smoothed images to generate another set of images → **Difference of Gaussians (DoG)**
- These DoG images are great for finding out interesting keypoints in the image
- DoG is obtained as the difference of Gaussian blurring of an image with two different, adjacent, σ (i.e., σ and $k\sigma$)
- Done for different octaves of the image

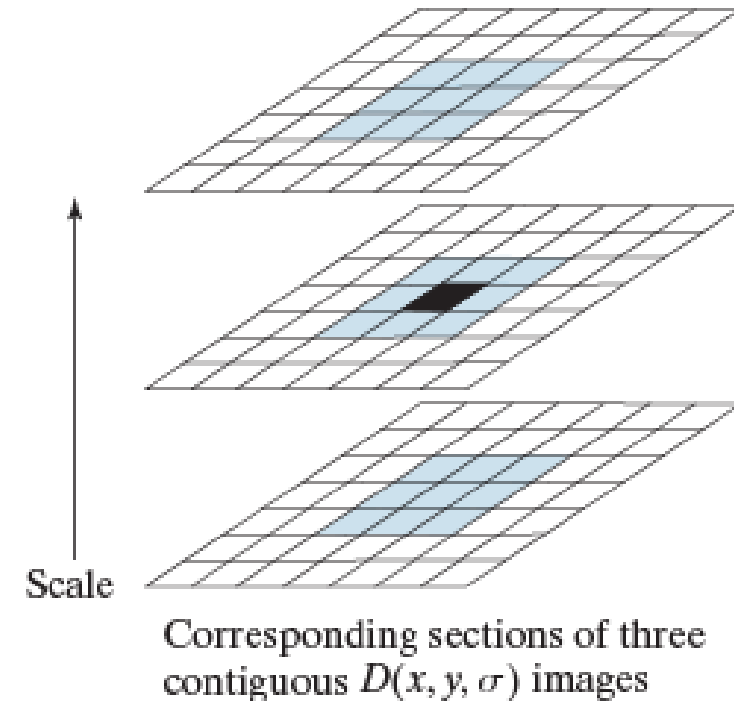


$$\begin{aligned} \mathbf{D}(x, y, \sigma) &= [G(x, y, k\sigma) - G(x, y, \sigma)] \star f(x, y) \\ &= L(x, y, k\sigma) - L(x, y, \sigma) \end{aligned}$$

SIFT: SCALE INVARIANT FEATURE TRANSFORM

Step 2: Keypoint localization (detecting local extrema)

- At each location in $D(x, y, \sigma)$, the value of the pixel is compared with its 8 neighbors, and the 9 neighbors above and below (26 checks are made)
- The point is selected as an extremum (maximum or minimum) point if its value is **larger** or **smaller** than the values of **all its neighbors**.
- No extrema can be detected in the first (last) scale of an octave because it has no lower (upper) scale image of the same size.



SIFT: SCALE INVARIANT FEATURE TRANSFORM

- Typically, a lot of **keypoints** are found
- Remove low contrast features, remove edges, etc.
- We will skip the details...



SIFT: SCALE INVARIANT FEATURE TRANSFORM

Step 3: Compute keypoint orientations

- **Assign a consistent orientation** to each keypoint based on local image properties → allows us to represent a keypoint relative to its own orientation and thus **achieve invariance to image rotation**
- The scale of the keypoint is used to select the Gaussian smooth image, L , corresponding to that scale. In this way, all orientation computations are performed in a scale-invariant manner
- A neighborhood is taken around the keypoint, and the **gradient magnitude** and **orientation** is calculated in that region using pixel differences:

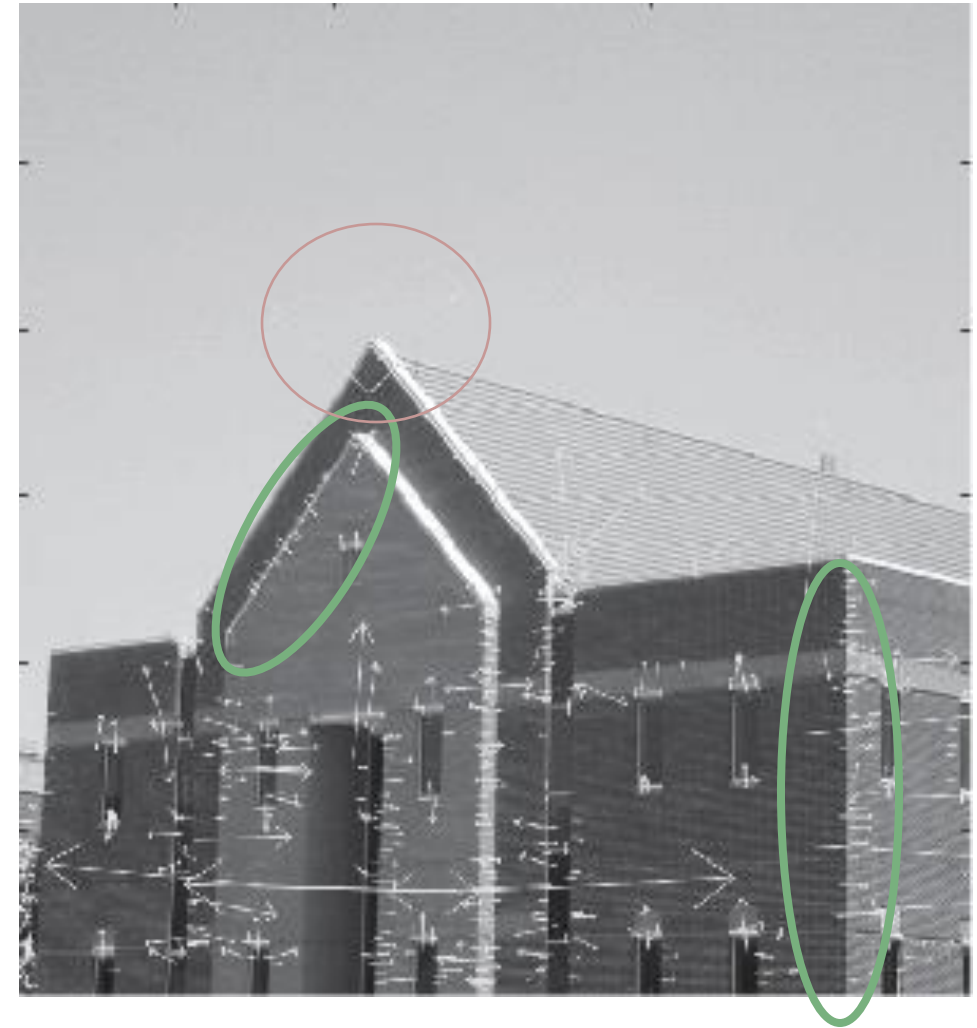
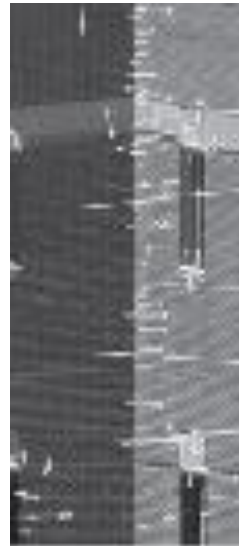
$$M(x, y) = [(L(x + 1, y) - L(x - 1, y))^2 + (L(x, y + 1) - L(x, y - 1))^2]^{\frac{1}{2}}$$
$$\theta(x, y) = \tan^{-1}[(L(x + 1, y) - L(x - 1, y)) / (L(x, y + 1) - L(x, y - 1))]$$

- An **orientation histogram** with 36 bins covering 360 degrees is created
- **Peaks** in the histogram **correspond to dominant local directions** of local gradients → assign it as the keypoint's orientation
- Improvement: the **highest peak** in the histogram is detected, and any other local peak that is within 80% of the highest peak is also used to create another keypoint with that orientation → it creates keypoints with **same location and scale, but different orientations**. This contributes to the stability of matching

SIFT: SCALE INVARIANT FEATURE TRANSFORM

Step 3: Compute keypoint orientations

- Arrows represent keypoint orientation
- Region surrounded in green: note the consistency of orientation of similar sets of keypoints. The lengths of arrows vary, depending on illumination and image content, but their **direction** is unmistakably **consistent**



SIFT: SCALE INVARIANT FEATURE TRANSFORM

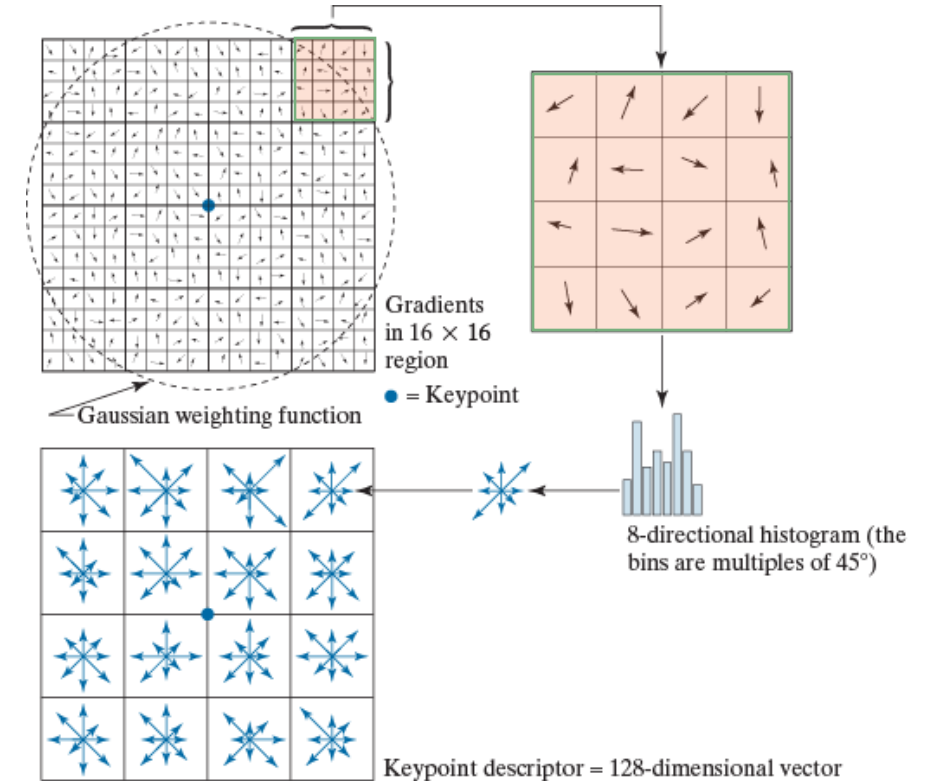
Step 4: Compute keypoint descriptor

- At this point, each keypoint is characterized by a **location** (x,y), **scale** (σ), and **orientation** (orientation histogram)
- Next is to compute a **descriptor** for a local region around each keypoint that is highly distinctive, AND, at the same time, as invariant as possible to changes in scale, orientation, illumination, and viewpoint
- The idea is to be able to use these descriptors to **identify matches** (similarities) between local regions in different images

SIFT: SCALE INVARIANT FEATURE TRANSFORM

Step 4: Compute keypoint descriptor

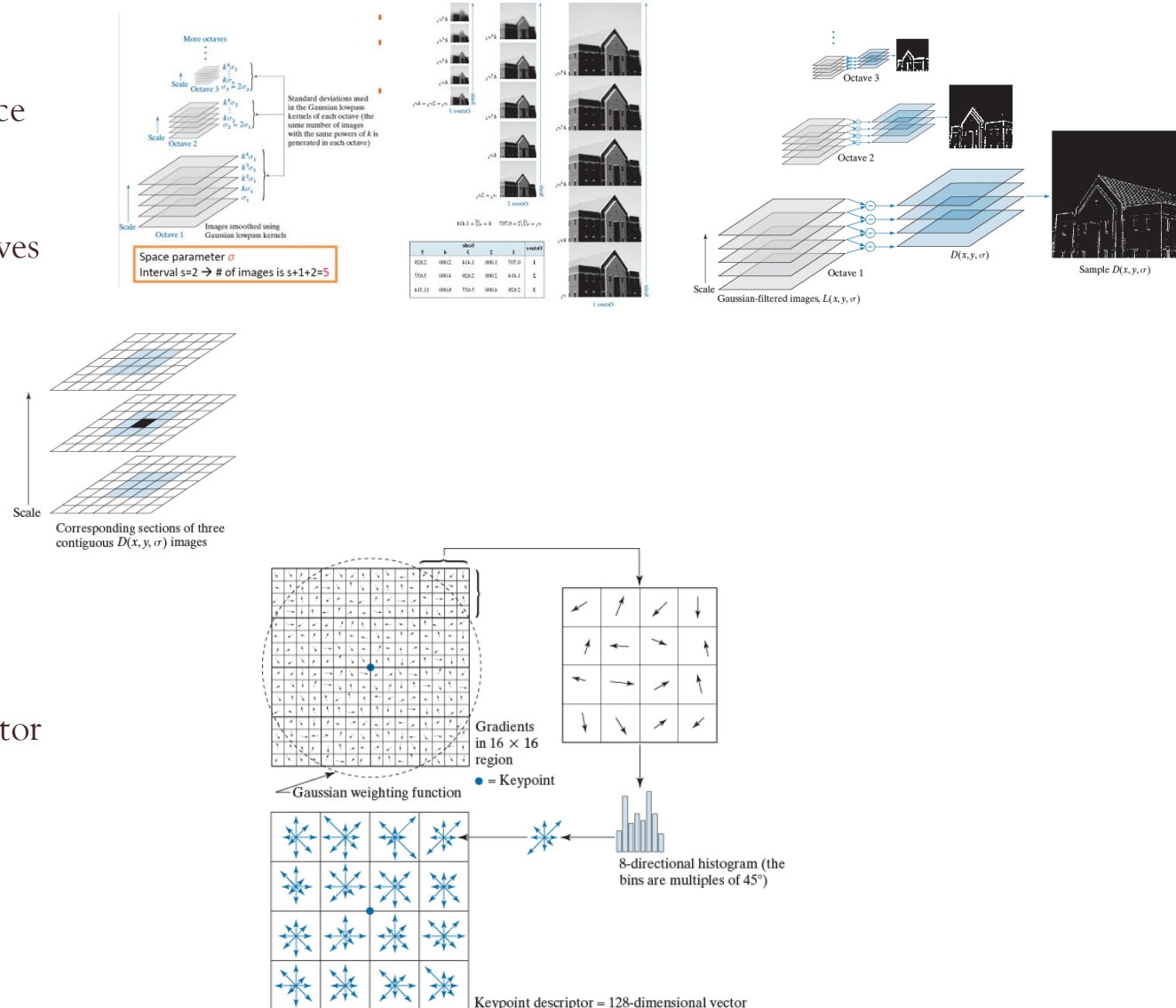
- A region of 16x16 pixels is centered on a keypoint
- The previously calculated dominant orientation is used to **rotate the patch**, ensuring rotation invariance
- The gradient magnitude and direction are computed at each point **using pixel differences**
- A Gaussian **weighting function** is used to assign a weight that multiplies the magnitude of the gradient at each point. More weight is given to the pixels near the center. The objective is to reduce sensitivity to noise and edges
- Division into 16 sub-blocks of 4x4 size
- For each sub-block, an 8 bin orientation histogram is created (quantize gradient orientations into 8 possible directions differing 45°)
- Total of 128 (4x4x8) values
- Vector form → keypoint descriptor
- In addition to this, several measures are taken to achieve robustness against illumination changes, rotation etc..



SIFT: SCALE INVARIANT FEATURE TRANSFORM

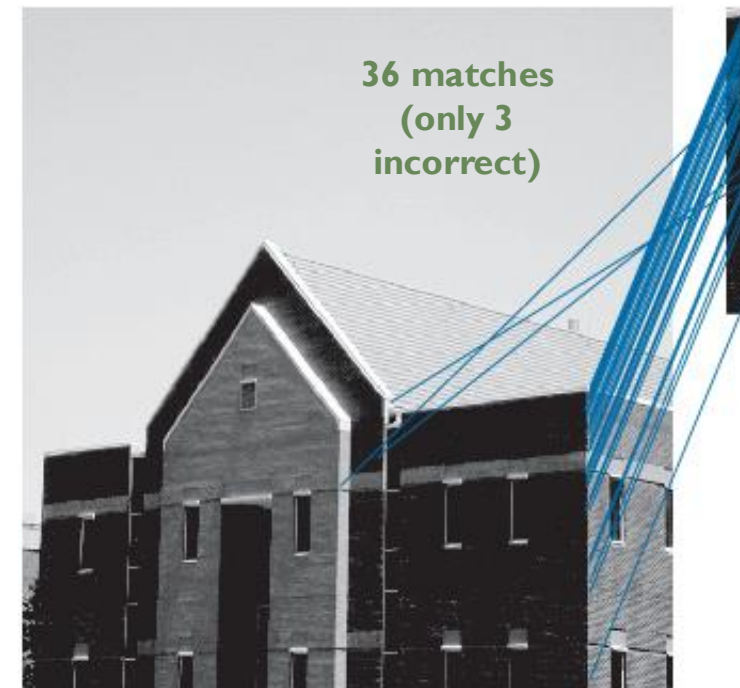
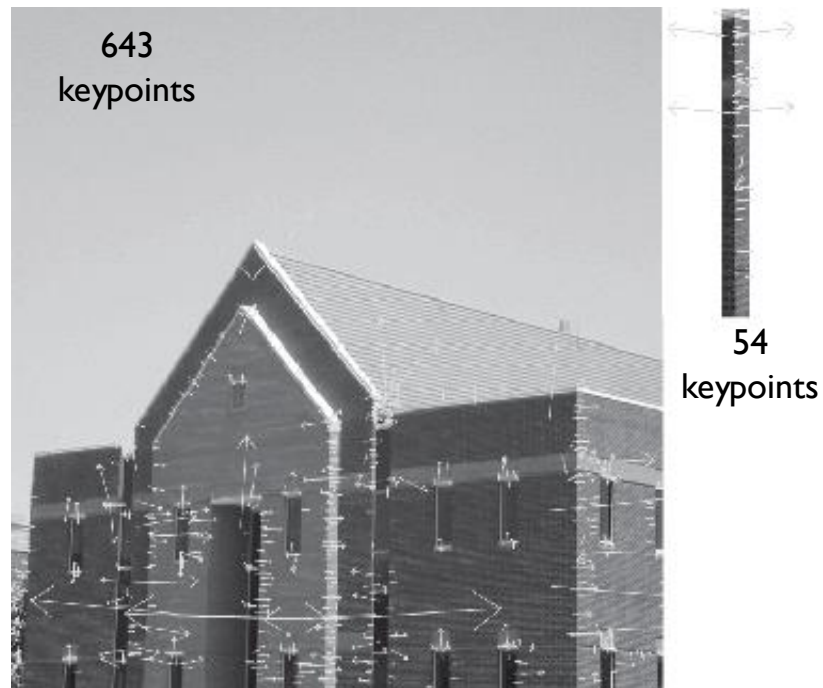
Summary of the SIFT algorithm:

1. Construct the **scale space** & compute Difference of Gaussians, DoG. Parameters: σ , s , (k is computed from s), and the number of octaves. Suggested values are $\sigma=1.6$, $s=2$, and three octaves
2. Key point localization (detecting local extrema)
3. Compute keypoint **orientations** (regional **gradient magnitude** and **direction**)
4. Compute keypoint **descriptors**. Compute a vector for each keypoint. Region of size 16×16 around each keypoint is used, the result will be a **128-dimensional feature vector** for each keypoint



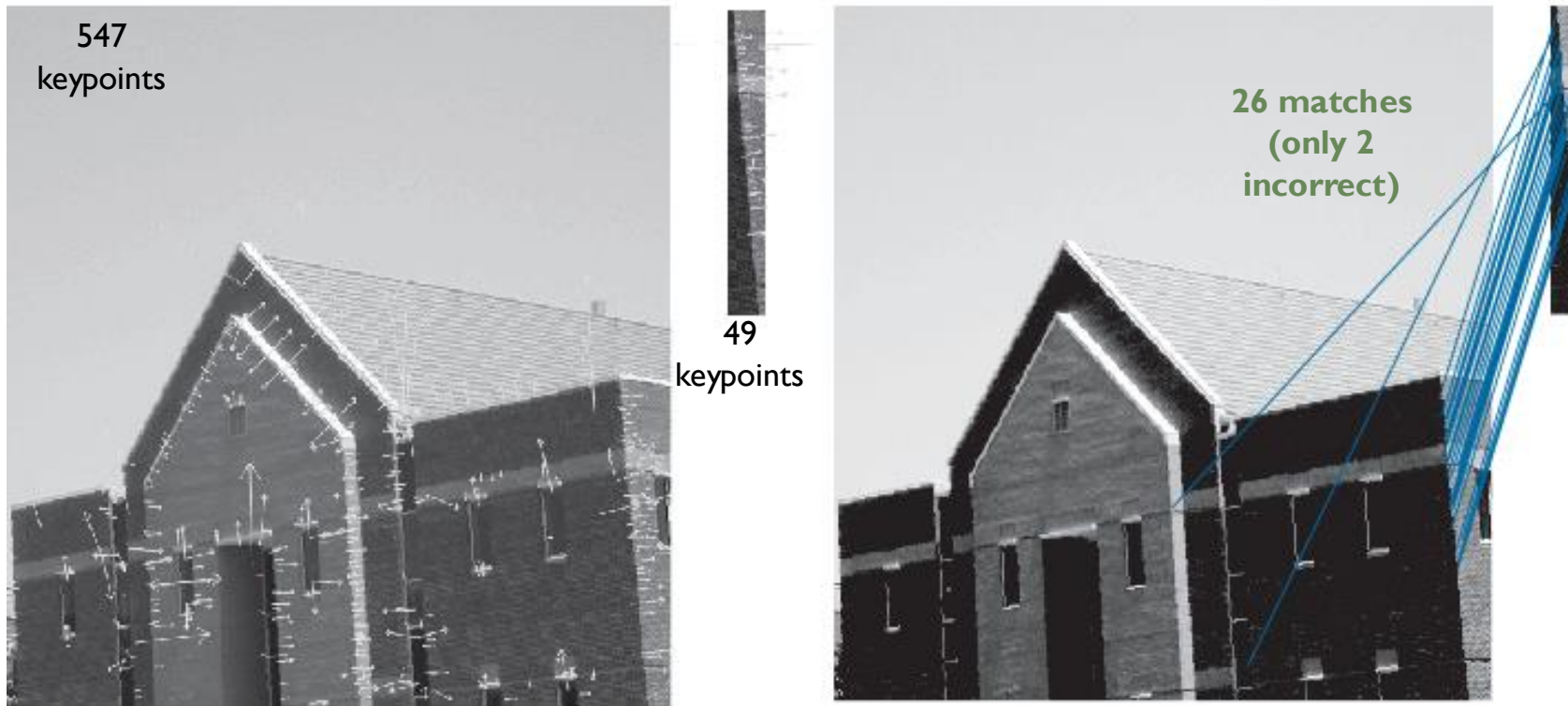
SIFT: SCALE INVARIANT FEATURE TRANSFORM

Example: find the number of matches between an image of a building and a subimage computed by extracting a piece of the right corner edge of the building



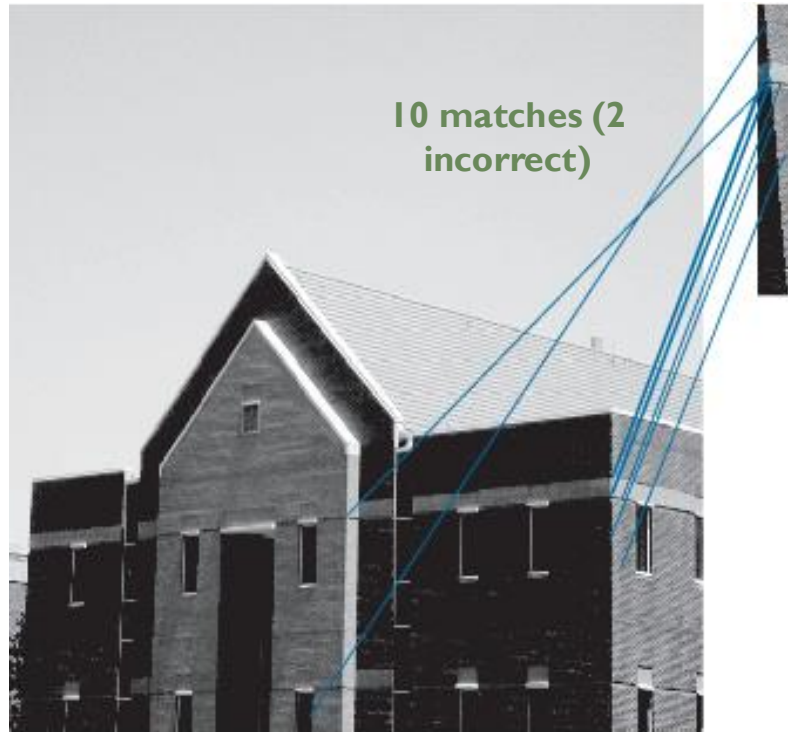
SIFT: SCALE INVARIANT FEATURE TRANSFORM

Example: same problem but after rotating 5°

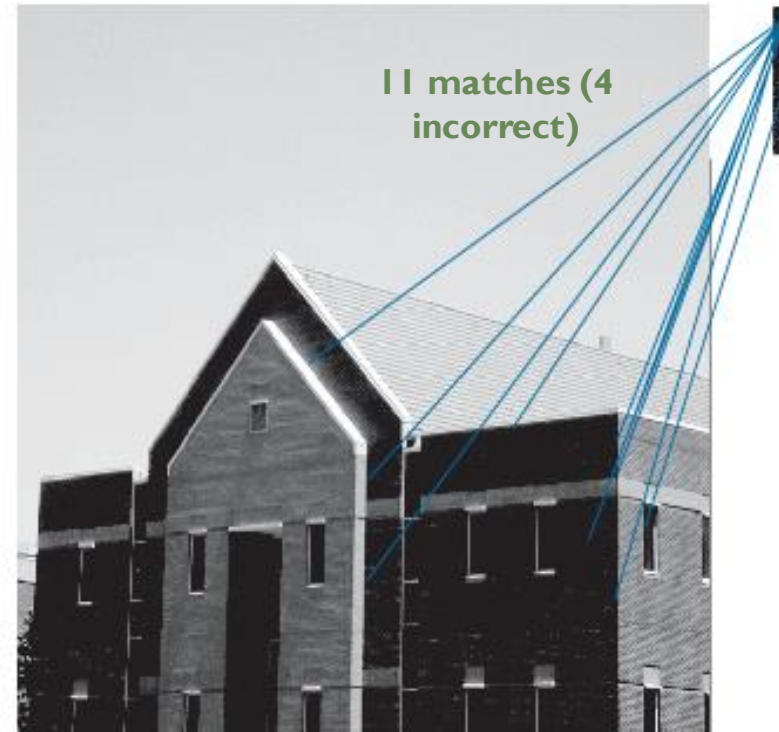


SIFT: SCALE INVARIANT FEATURE TRANSFORM

Example: maybe more realistic cases. Original image vs rotated subimage; Original image vs half-sized subimage



Original building image vs
rotated subimage



Original building image vs
half-sized subimage